

UNIVERSITÉ DE KINSHASA

Faculté des Sciences et Technologies

Mention Mathématiques, Statistiques et Informatique



TRAVAIL PRATIQUE DE SCIENCE DES DONNÉES

Cours : Science de données et Big Data

IMPLÉMENTATION DE L'ALGORITHME DES NUÉES DYNAMIQUES (FROM SCRATCH)

Présenté par :

— LUMEYA KWIVANGANA EXAUCÉE

Promotion :

— MASTER 1 INTELLIGENCE ARTIFICIELLE

Professeur :

— KAFUNDA JEAN-PIERRE

Assisté par :

— GRADY KAMINGU

Année Académique : 2025–2026

Table des matières

1	Introduction	3
1.1	Objectifs	3
2	Fondements théoriques	4
2.1	Le problème de classification	4
2.2	Principe itératif des Nuées dynamiques	4
2.3	Les étalons dans l'article original	5
3	Relation avec K-means	5
4	Les représentations proposées dans l'application	6
4.1	Représentation par un point	6
4.2	Ensemble de points représentatifs	6
4.3	Représentation par axes factoriels	7
4.4	Représentation par distribution probabiliste	7
4.5	Structure représentative	8
5	Formalisation de l'algorithme implémenté	8
5.1	Étape 1 : initialisation	8
5.2	Étape 2 : affectation	9
5.3	Étape 3 : reconstruction	9
5.4	Étape 4 : critère	9
5.5	Étape 5 : convergence	9
6	Architecture logicielle	10
7	Implémentation from scratch	11
8	Jeu de données expérimental	11
9	Résultats expérimentaux	12
9.1	Points représentatifs	13

9.2	Axes factoriels	14
9.3	Distribution probabiliste	15
9.4	Structure représentative	16
9.5	Cas point : K-means	17
10	Analyse et discussion	18
10.1	Influence de la représentation	18
10.2	Influence de l'initialisation	18
10.3	Classes vides	18
10.4	Limites	19
11	Tests et reproductibilité	19
12	Utilisation du projet	19
13	Conclusion	20

1 Introduction

La classification non supervisée consiste à organiser un ensemble d'individus en groupes homogènes sans disposer, au départ, d'une étiquette de classe connue pour chaque individu. L'objectif général est de construire une partition dans laquelle les individus d'une même classe se ressemblent davantage que ceux appartenant à des classes différentes.

Dans son article fondateur publié en 1971 dans la *Revue de Statistique Appliquée*, Edwin Diday présente la **méthode des nuées dynamiques** comme une méthode de classification automatique et de reconnaissance des formes. L'article commence précisément par le problème de génération d'une partition à partir d'un corps de données, avec les objectifs de maximiser la ressemblance à l'intérieur des parties et de la minimiser entre parties différentes [1].

Le principe est itératif. Une représentation est associée à chaque classe, les distances sont calculées, une nouvelle partition est construite, puis les représentations sont reconstruites. L'article formalise cette démarche dans la section consacrée à la méthode et à l'algorithme [1].

Le présent travail vise à implémenter ce principe **from scratch**. L'application développée ne se limite pas au cas d'un centroïde : elle permet de sélectionner plusieurs familles de représentations demandées dans l'énoncé du TP : points représentatifs, axes factoriels, distribution probabiliste, structure représentative et point, ce dernier correspondant au cas K-means.

1.1 Objectifs

Les objectifs sont les suivants :

- comprendre le principe des Nuées dynamiques ;
- formaliser l'alternance entre partition et représentation ;
- implémenter l'algorithme sans utiliser de fonction de clustering spécialisée ;
- montrer le cas particulier où la représentation est un point ;
- proposer plusieurs représentations de classe ;
- produire des visualisations et des mesures de convergence ;
- fournir un programme reproductible et documenté ;
- fournir un rapport LaTeX et un dépôt GitHub cohérents avec l'implémentation.

2 Fondements théoriques

2.1 Le problème de classification

Soit E l'ensemble des objets à classer. L'article de Diday introduit également P comme l'ensemble des parties de E , ainsi qu'une fonction D permettant de mesurer une distance ou une dissimilarité entre un objet et une partie [1, p. 20].

Dans notre implémentation, chaque observation est représentée par un vecteur numérique :

$$x_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \in \mathbb{R}^p,$$

avec $i \in \{1, \dots, n\}$.

On recherche une partition :

$$P = \{C_1, C_2, \dots, C_K\},$$

avec :

$$\bigcup_{j=1}^K C_j = E, \quad C_j \cap C_l = \emptyset \quad \text{pour } j \neq l.$$

2.2 Principe itératif des Nuées dynamiques

Diday note une configuration de classes par un élément L de l'espace des configurations. Pour passer d'une configuration à la suivante, l'algorithme commence par calculer les distances entre les objets et les représentations de classes, puis construit une partition, avant de construire les nouveaux ensembles représentatifs [1, p. 21].

Le schéma retenu dans notre application est :

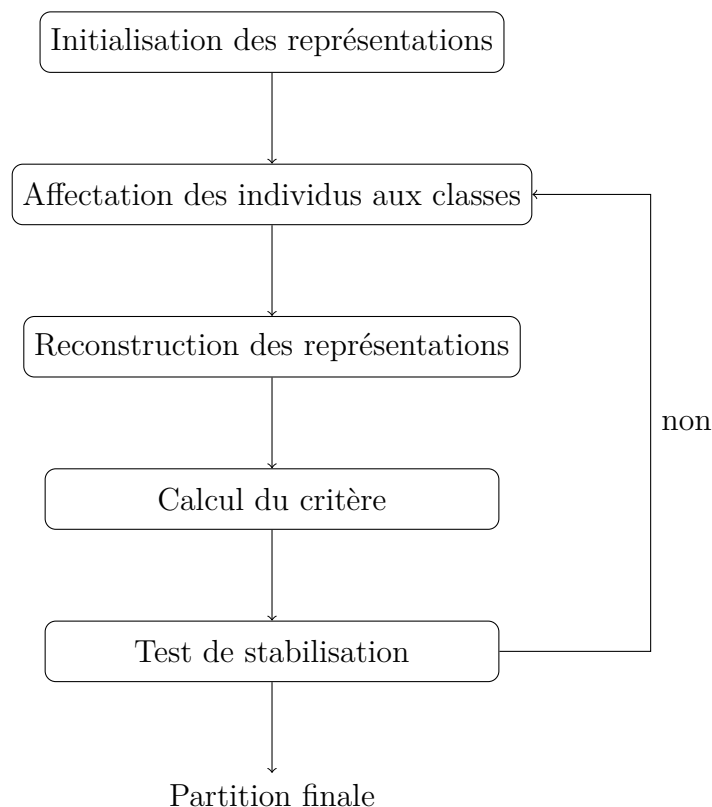


FIGURE 1 – Schéma général de l’algorithme implémenté.

2.3 Les étalons dans l’article original

L’article de 1971 introduit explicitement les éléments d’un ensemble E_i comme des *étalons* de la classe correspondante. Dans la discussion, Diday souligne que plusieurs étalons peuvent constituer un « squelette » ou une sorte d’axe factoriel discret d’une forme, et montre sur un exemple que l’utilisation de plusieurs étalons permet de mieux conserver certaines formes allongées qu’un simple centre de gravité [1, p. 25].

Cette observation est particulièrement importante pour le présent TP : elle justifie conceptuellement l’intérêt de ne pas réduire systématiquement une classe à un unique point.

3 Relation avec K-means

Le K-means représente chaque classe par un seul centroïde. Pour une classe C_j , son prototype est :

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i.$$

La distance utilisée dans notre cas est :

$$d(x, \mu_j) = \|x - \mu_j\|_2^2.$$

La fonction objectif correspondante est :

$$J_{\text{KM}} = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|_2^2.$$

Dans notre architecture, le cas K-means n'est donc pas un programme séparé. Il correspond au choix :

$$R_j = \mu_j.$$

Le même moteur itératif peut ainsi fonctionner avec plusieurs types de prototypes. Lorsque le prototype est réduit à un point, on obtient le cas K-means utilisé dans l'application.

4 Les représentations proposées dans l'application

L'énoncé demande à l'utilisateur de choisir la représentation de la nuée. Les cinq choix ont été implémentés de manière explicite.

4.1 Représentation par un point

C'est le cas K-means présenté précédemment :

$$R_j = \mu_j.$$

Il s'agit de la représentation la plus compacte, mais elle ne conserve que la position centrale de la classe.

4.2 Ensemble de points représentatifs

La classe est représentée par q observations :

$$R_j = \{r_{j1}, r_{j2}, \dots, r_{jq}\}.$$

La distance d'un individu x à la classe est définie par :

$$d(x, R_j) = \min_{r \in R_j} \|x - r\|_2^2.$$

Le programme sélectionne les représentants à partir des observations de la classe. Le premier représentant est un médoïde, puis les suivants sont choisis par une procédure

gloutonne qui cherche à réduire la distance de couverture des observations par l'ensemble des représentants.

Cette construction est une implémentation opérationnelle du principe d'étalons ; elle ne prétend pas reproduire toutes les variantes de fonction d'agrégation-écartement étudiées dans l'article original.

4.3 Représentation par axes factoriels

Pour une classe C_j , on calcule sa covariance :

$$S_j = \frac{1}{|C_j| - 1} \sum_{x_i \in C_j} (x_i - \mu_j)(x_i - \mu_j)^T.$$

On recherche ensuite les couples propres :

$$S_j v_r = \lambda_r v_r.$$

Les vecteurs propres associés aux plus grandes valeurs propres donnent les directions principales. La représentation est :

$$R_j = (\mu_j, V_j).$$

Pour mesurer l'écart à l'axe ou au sous-espace, on projette $x - \mu_j$ sur le sous-espace engendré par les axes retenus et on mesure la norme du résidu :

$$d(x, R_j) = \|(x - \mu_j) - P_j(x - \mu_j)\|_2^2.$$

Dans un problème bidimensionnel, le programme limite automatiquement le nombre d'axes à un sous-espace non trivial.

4.4 Représentation par distribution probabiliste

Dans ce mode, la classe est caractérisée par une moyenne et une covariance :

$$R_j = (\mu_j, \Sigma_j).$$

On utilise la distance de Mahalanobis au carré :

$$d(x, R_j) = (x - \mu_j)^T \Sigma_j^{-1} (x - \mu_j).$$

Pour éviter les difficultés liées à une matrice de covariance singulière, le programme utilise :

$$\Sigma_j^* = \Sigma_j + \varepsilon I,$$

avec une petite valeur ε .

Cette représentation prend donc en compte non seulement la position centrale, mais également la dispersion et les corrélations entre variables.

4.5 Structure représentative

Ce mode constitue une représentation composite adaptée à l'application. La classe est décrite par :

$$R_j = (\mu_j, \Sigma_j, E_j),$$

où E_j est un ensemble d'étalons.

La distance combine trois informations :

1. distance au centre ;
2. distance de Mahalanobis ;
3. distance au représentant le plus proche.

Les composantes sont normalisées avant combinaison afin qu'une seule mesure ne domine artificiellement la distance composite.

Remarque importante : cette définition constitue un choix d'ingénierie pour donner un sens opérationnel au terme « structure représentative » dans l'application demandée par le TP. L'article de Diday présente un cadre général d'agrégation-écartement et des exemples de choix de fonctions ; il ne fournit pas, dans les pages étudiées, une unique définition universelle portant exactement ce nom.

5 Formalisation de l'algorithme implémenté

Soit $R_1, \dots, R_K$ l'ensemble des représentations courantes.

5.1 Étape 1 : initialisation

Le programme tire aléatoirement K individus distincts du jeu de données et les utilise comme points de départ. Les observations sont ensuite affectées à la représentation ponctuelle initiale la plus proche.

Ce choix est cohérent avec l'article de Diday, qui indique qu'en l'absence d'information a priori, les étalons peuvent être tirés automatiquement au hasard [1, p. 23].

Une graine aléatoire (‘seed’) est fixée par défaut à 42 afin de rendre l’expérience reproductible.

5.2 Étape 2 : affectation

Pour chaque observation x_i et chaque représentation R_j , le programme calcule :

$$d_{ij} = d(x_i, R_j).$$

L’observation est affectée à :

$$\ell_i = \arg \min_j d(x_i, R_j).$$

5.3 Étape 3 : reconstruction

Une fois les classes obtenues, le programme reconstruit chaque représentation :

- moyenne pour le mode point ;
- étalons pour le mode points ;
- moyenne + vecteurs propres pour les axes ;
- moyenne + covariance pour la distribution ;
- centre + covariance + étalons pour la structure.

5.4 Étape 4 : critère

Le programme utilise le critère :

$$J = \sum_{i=1}^n \min_j d(x_i, R_j).$$

Ce critère est adapté à l’implémentation pédagogique des cinq représentations. Il faut cependant distinguer cette définition opérationnelle du cadre général de Diday, dans lequel une fonction $R(x, i, L)$ d’agrégation-écartement peut être choisie selon le problème [1, p. 21].

5.5 Étape 5 : convergence

La stabilisation est déclarée lorsqu’une des conditions suivantes est satisfaite :

1. les affectations ne changent plus ;
2. la variation relative du critère devient inférieure à la tolérance ;

3. le nombre maximal d'itérations est atteint.

L'article original donne une étude théorique de convergence et introduit la notion d'optimum local ; il indique notamment que, sous certaines propriétés du critère, la suite converge vers un optimum local [1, p. 22–23].

Notre test de stabilisation est une version numérique et pratique destinée au programme du TP.

6 Architecture logicielle

Le projet est organisé afin de séparer l'algorithme, l'interface, les données, les tests et le rapport :

```
nuees_project/  
|-- main.py  
|-- setup_environment.py  
|-- requirements.txt  
|-- src/  
|   '-- dynamic_clouds.py  
|-- data/  
|-- tests/  
|-- results/  
'-- rapport/
```

La classe principale est 'DynamicClouds'. Elle reçoit notamment :

- K : nombre de classes ;
- 'mode' : représentation choisie ;
- q : nombre d'étalons ou d'axes demandé ;
- 'max_iter' : nombre maximal d'itérations ;
- 'tol' : tolérance ;
- 'seed' : graine de reproductibilité.

Le choix du prototype est encapsulé dans des structures distinctes : 'PointPrototype', 'PointsPrototype', 'AxesPrototype', 'DistributionPrototype' et 'StructurePrototype'.

Cette architecture permet de conserver un moteur d'affectation commun tout en changeant la représentation et la distance.

7 Implémentation from scratch

L'implémentation n'utilise pas 'sklearn.cluster.KMeans' ni une autre fonction de clustering externe. NumPy est utilisé pour les calculs matriciels, Pandas pour le chargement des données et Matplotlib pour les visualisations.

Un extrait représentatif du moteur est :

Listing 1 – Schéma de l'itération principale.

```
1 for iteration in range(1, max_iter + 1):
2     prototypes = build_prototypes(X, labels)
3     new_labels, distances = assign(X, prototypes)
4     criterion = sum(min(row) for row in distances)
5
6     if labels_are_stable(labels, new_labels):
7         break
8
9     labels = new_labels
```

Les opérations mathématiques utilisées pour les distances et les covariances sont écrites directement avec NumPy.

8 Jeu de données expérimental

Pour permettre une démonstration reproductible sans dépendre d'un fichier externe, un jeu de données synthétique bidimensionnel est fourni.

Il comporte 400 observations et quatre groupes générés à partir de distributions normales multivariées ayant des centres et des matrices de covariance différents. L'objectif n'est pas de reproduire un jeu de données réel, mais de créer des formes suffisamment différentes pour observer l'effet de la représentation choisie.

Le programme standardise les variables avant la classification lorsque le jeu de données est chargé depuis un CSV.

TABLE 1 – Paramètres de l’expérience de démonstration.

Paramètre	Valeur
Nombre d’observations	400
Nombre de variables	2
Nombre de classes demandé	4
Nombre d’étalons/axes demandé	3
Graine aléatoire	42
Nombre maximal d’itérations	50

9 Résultats expérimentaux

L’expérience complète peut être reproduite par :

```
python main.py --mode all --k 4 --q 3 --seed 42
```

Les résultats produits lors de la préparation de ce rapport sont synthétisés ci-dessous.

TABLE 2 – Résumé de l’exécution sur le jeu de données de démonstration.

Représentation	Itérations	Critère final	Tailles des classes
Points	3	258.596973	101 / 100 / 99 / 100
Axes	16	85.662361	119 / 95 / 89 / 97
Distribution	3	791.998525	103 / 100 / 97 / 100
Structure	3	16.835066	101 / 100 / 99 / 100
Point / K-means	3	717.700277	101 / 100 / 100 / 99

Attention à l’interprétation : les valeurs du critère ne peuvent pas être utilisées pour classer directement les cinq représentations. Le critère repose sur des distances différentes selon le prototype : euclidienne, distance à un sous-espace, Mahalanobis ou combinaison de plusieurs composantes. Le tableau sert donc à vérifier la convergence et la reproductibilité de l’implémentation, pas à établir un classement entre méthodes.

9.1 Points représentatifs

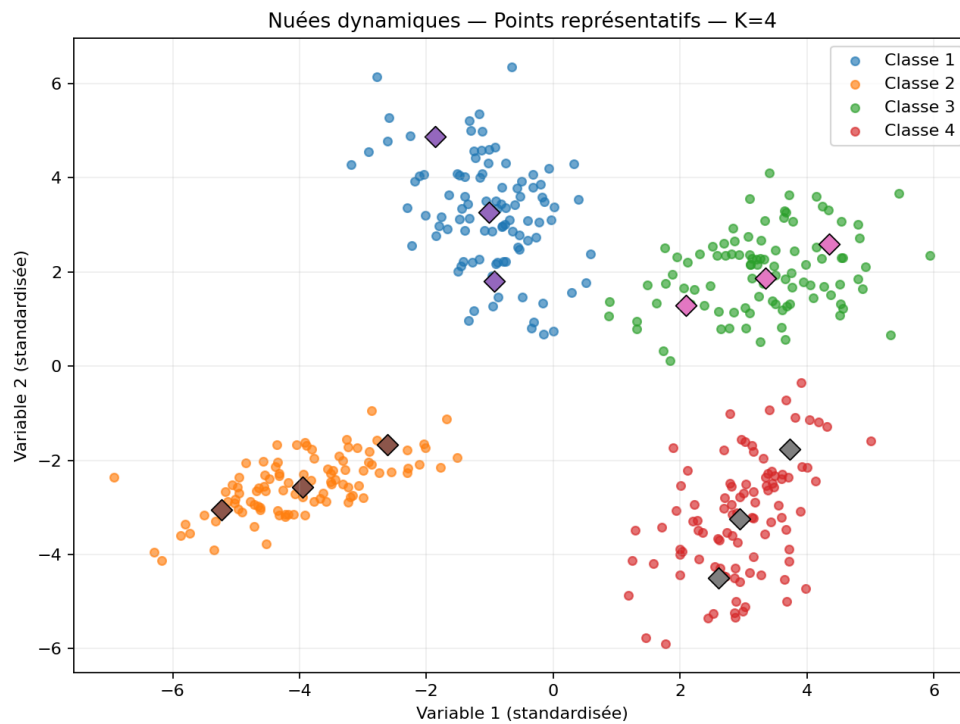


FIGURE 2 – Partition obtenue avec plusieurs points représentatifs par classe.

La figure permet d'observer les individus ainsi que les représentants utilisés pour caractériser chaque groupe.

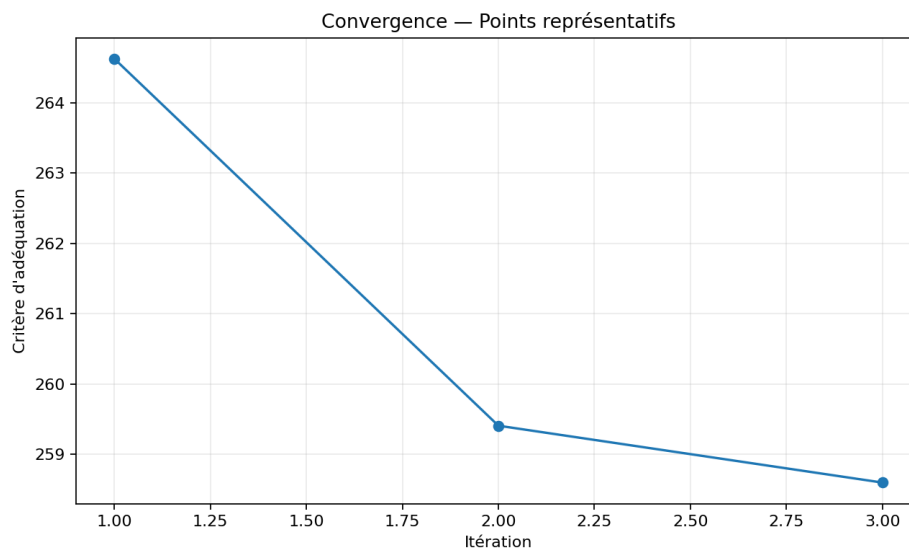


FIGURE 3 – Évolution du critère pour la représentation par points.

9.2 Axes factoriels

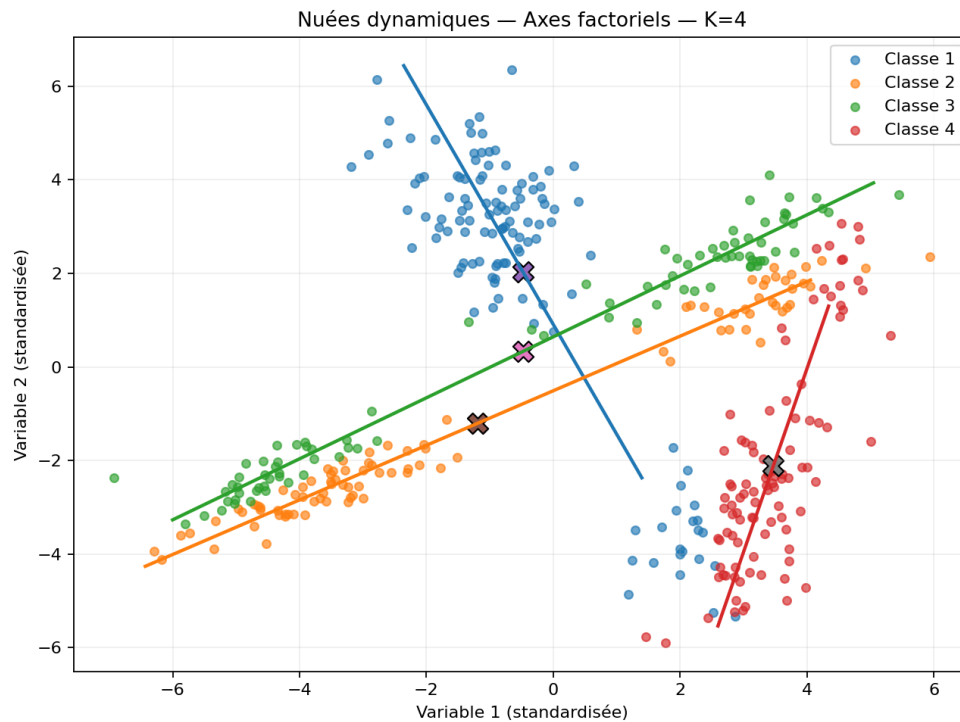


FIGURE 4 – Partition et axes principaux des classes.

Ce mode cherche à conserver une structure directionnelle. Il est donc particulièrement intéressant lorsque la géométrie d'une classe n'est pas correctement résumée par un seul centre.

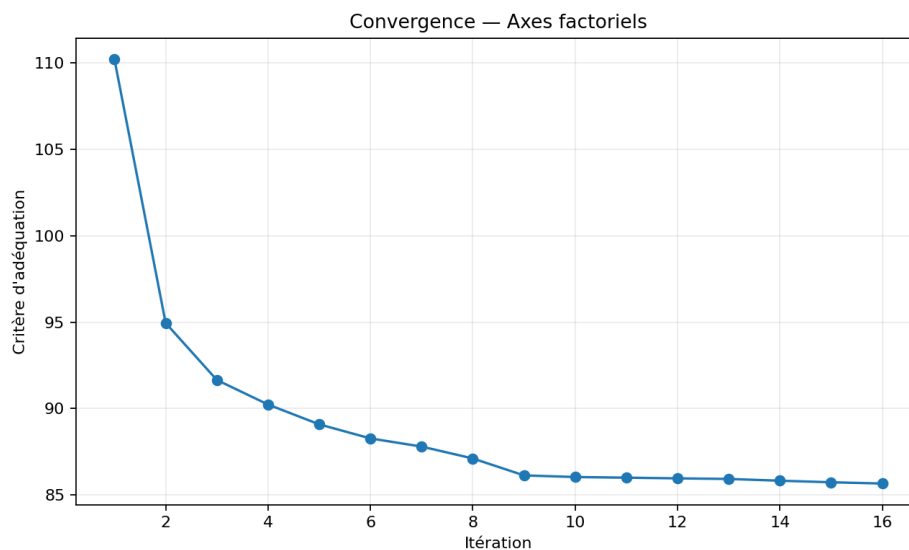


FIGURE 5 – Convergence de la représentation par axes.

9.3 Distribution probabiliste

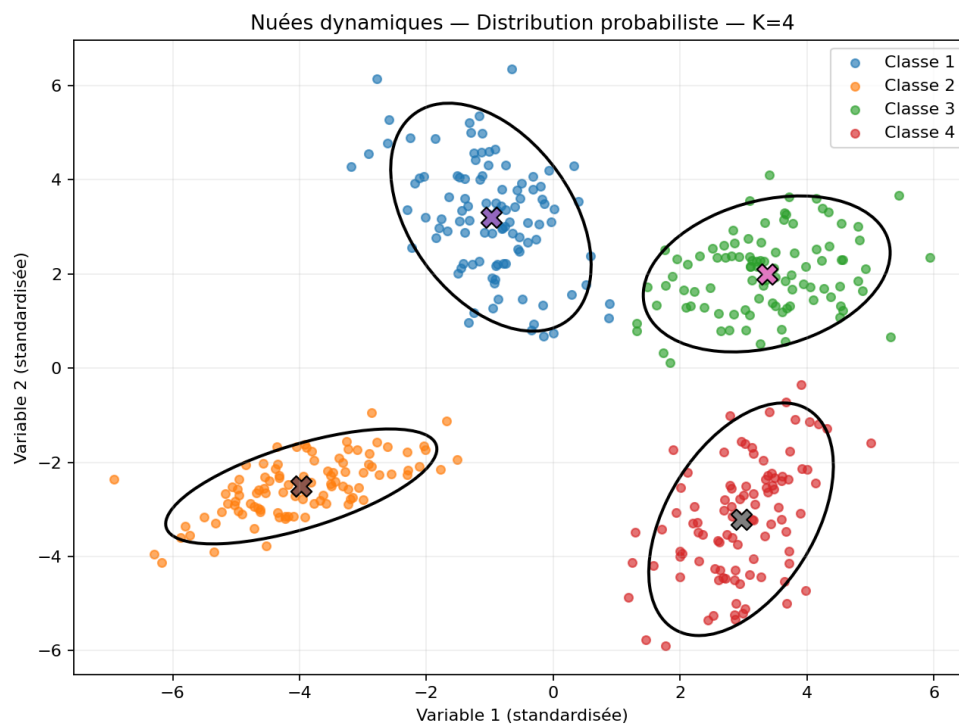


FIGURE 6 – Partition avec une représentation moyenne-covariance. Les ellipses visualisent la dispersion.

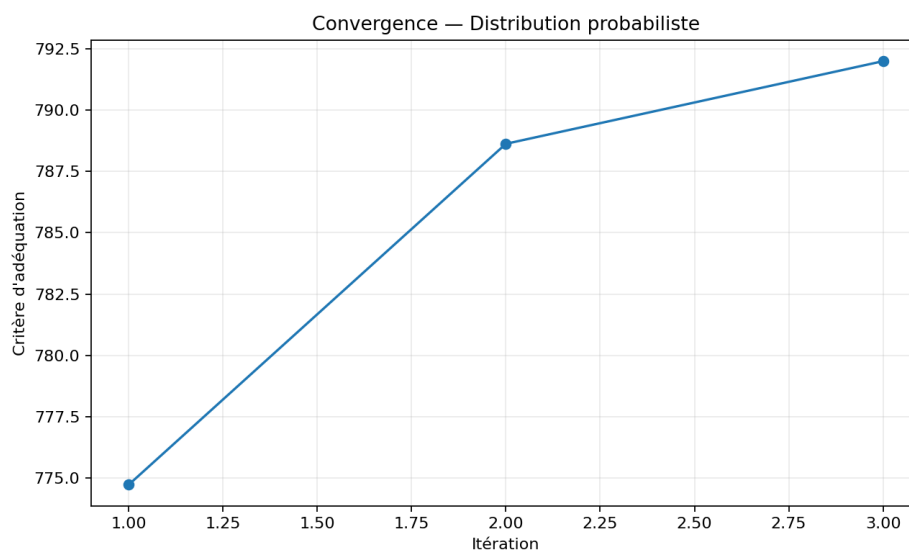


FIGURE 7 – Convergence de la représentation probabiliste.

9.4 Structure représentative

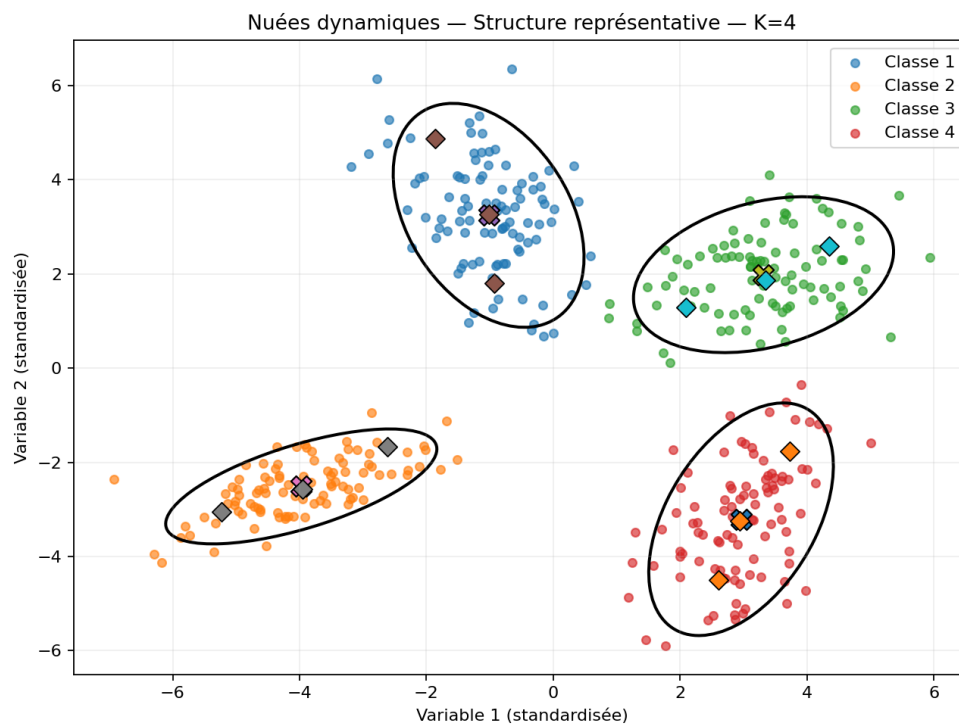


FIGURE 8 – Structure représentative : centre, dispersion et étalons.

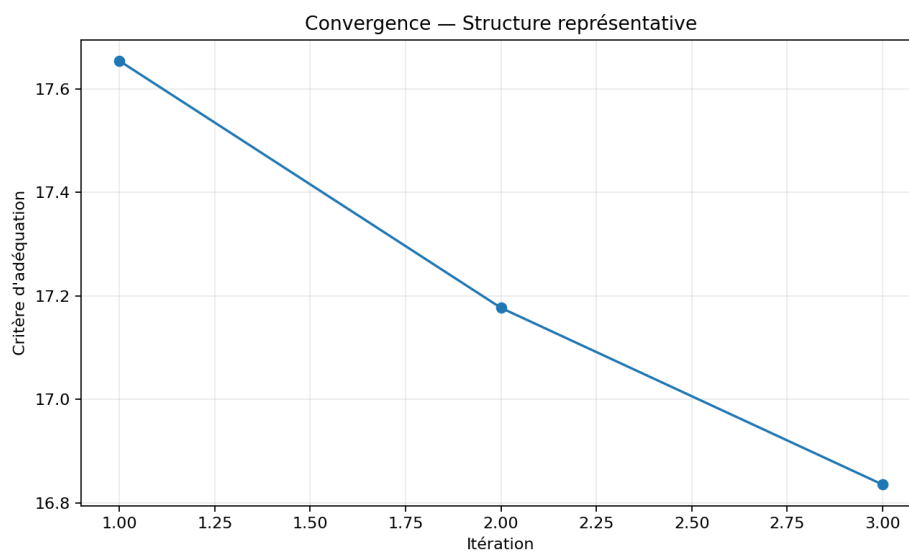


FIGURE 9 – Convergence de la structure représentative.

9.5 Cas point : K-means

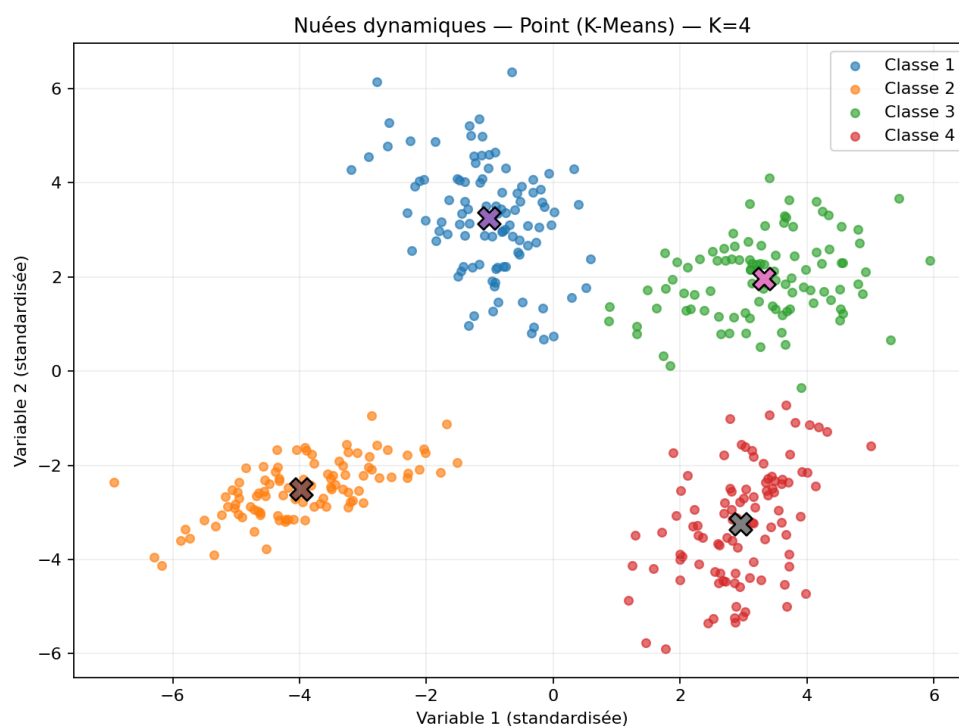


FIGURE 10 – Cas où chaque classe est représentée par un seul point.

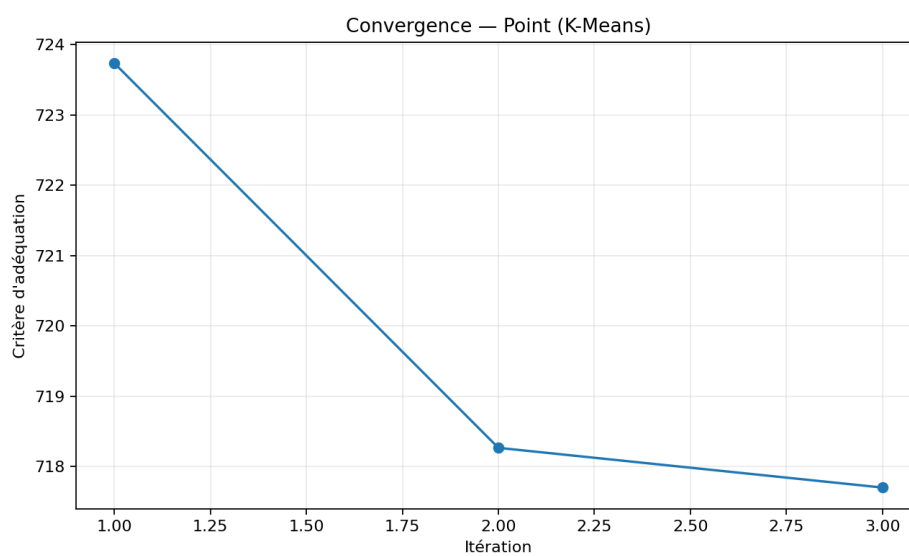


FIGURE 11 – Convergence du cas K-means.

10 Analyse et discussion

10.1 Influence de la représentation

Le principal intérêt de l'application est de montrer qu'une même partition peut être abordée à partir de prototypes de nature différente. Un point résume essentiellement la position moyenne. Plusieurs étalons apportent une information spatiale plus riche. Les axes apportent une information directionnelle, tandis que la covariance introduit une information sur la dispersion et les corrélations.

Cette distinction est cohérente avec l'idée développée par Diday dans l'exemple artificiel de l'article : l'auteur observe que plusieurs étalons peuvent former un squelette de la forme et que le seul centre de gravité peut perdre l'information correspondant à des formes allongées [1, p. 25].

10.2 Influence de l'initialisation

Comme toute méthode itérative de partitionnement de ce type, l'initialisation peut influencer le résultat final. L'article de Diday insiste d'ailleurs sur l'intérêt d'effectuer plusieurs tirages de départ : lorsque des groupes similaires apparaissent de manière répétée, on peut obtenir une information sur la classifiabilité des données [1, p. 24].

Le programme utilise une graine afin de rendre l'expérience reproductible. Pour étudier l'influence de l'initialisation, il suffit de modifier ‘--seed’.

Exemple :

```
python main.py --mode points --k 4 --q 3 --seed 7
python main.py --mode points --k 4 --q 3 --seed 21
python main.py --mode points --k 4 --q 3 --seed 42
```

Une extension naturelle du TP serait de répéter automatiquement l'algorithme plusieurs fois et de comparer les partitions obtenues.

10.3 Classes vides

Une classe peut devenir vide après une affectation. L'article original signale également l'apparition de classes vides lorsque le nombre de classes demandé est trop grand par rapport au nombre de groupes réellement présents [1, p. 25].

Dans notre implémentation, une classe vide est réparée en réaffectant à cette classe un individu provenant d'une classe non singleton. Cette stratégie permet au moteur de poursuivre l'itération sans abandonner l'expérience.

10.4 Limites

Les principales limites de cette implémentation sont :

- les fonctions de distance sont volontairement simples ;
- la représentation probabiliste suppose une structure moyenne-covariance ;
- la structure représentative est une définition opérationnelle construite pour le TP ;
- l'algorithme n'explore pas automatiquement toutes les valeurs possibles de K ;
- l'évaluation n'utilise pas ici de vérité terrain externe ;
- la visualisation est directement adaptée aux deux premières variables pour les données de dimension supérieure.

Ces limites ne remettent pas en cause l'objectif pédagogique principal : construire un moteur de classification dynamique dans lequel la nature du prototype peut être modifiée.

11 Tests et reproductibilité

Deux tests automatisés sont fournis. Ils vérifient :

1. que les cinq représentations peuvent être entraînées sur le jeu de données de démonstration ;
2. que le cas point construit bien un prototype par classe et permet la prédiction.

La commande est :

```
python -m unittest discover -s tests -v
```

Lors de la préparation du dépôt, les tests ont donné :

```
Ran 2 tests  
OK
```

La reproductibilité est obtenue en fixant `extttseed=42` dans les expériences de référence.

12 Utilisation du projet

Le dépôt contient un script `extttsetup_environment.py`. Contrairement à un simple appel systématique à `extttpip install -r requirements.txt`, ce script vérifie d'abord la présence et la version des bibliothèques nécessaires. Si elles sont déjà satisfaisantes, aucun téléchargement n'est effectué.

Pour lancer le projet :

```
python setup_environment.py  
python main.py
```

Ou, directement sous Windows :

```
run_windows.bat
```

Le fichier ‘README.md’ contient toutes les commandes détaillées ainsi que la procédure pour utiliser un autre fichier CSV.

13 Conclusion

Ce travail a permis de réaliser une implémentation from scratch du principe des Nuées dynamiques en construisant un moteur générique capable de fonctionner avec plusieurs représentations de classes.

Le point essentiel est l’alternance entre l’affectation des individus et la reconstruction des représentations. Dans le cas le plus simple, la représentation est réduite à un point et l’on retrouve le principe du K-means. En revanche, l’utilisation de plusieurs étalons, d’axes, d’une covariance ou d’une structure composite permet de conserver davantage d’information sur la forme d’une classe.

Le travail est directement inspiré de la formulation de Diday : l’article de 1971 définit le problème de partitionnement, formalise les configurations, décrit l’algorithme itératif et étudie la convergence. Il met également en évidence l’intérêt des étalons pour représenter des formes qui seraient trop grossièrement résumées par un seul centre [1].

L’application réalisée constitue donc une base pédagogique permettant d’expérimenter concrètement l’idée centrale du TP : **la Nuée dynamique fournit un cadre général dans lequel la représentation d’une classe peut être adaptée au problème étudié.**

Références

- [1] E. Diday, *Une nouvelle méthode en classification automatique et reconnaissance des formes : la méthode des nuées dynamiques*, Revue de Statistique Appliquée, tome 19, n° 2, 1971, pp. 19–33. Disponible sur NumDAM : https://www.numdam.org/item/RSA_1971__19_2_19_0/.